

■ GRASP — 9 Princípios (5 questões)	■ Coleções em Java — 5 questões
1. Information Expert ← mais cobrado Atribuir responsabilidade à classe que TEM a informação. Ex: Venda calcula total (tem os itens).	Hierarquia: Iterable → Collection → List , Set , Queue Map NÃO herda de Collection ← cilada clássica!
2. Creator B cria A se B: agrega/contém A · usa A · tem dados iniciais de A · registra A. Ex: Venda cria LinhaDeItem .	List — ordem garantida por índice: ArrayList : O(1) por índice. Mantém ordem de inserção. ✓ LinkedList : O(1) inserção. Também é Queue/Deque.
3. Controller Recebe eventos de sistema da UI. Opções: fachada do sistema (sistema todo) ou caso de uso (cenário). NÃO é objeto de UI. Ex: GerenciadorDeVenda .	Set — sem duplicatas: HashSet : unordered (sem ordem de iteração). O(1). LinkedHashSet : ordered (mantém ordem de inserção). TreeSet : sorted (ordem natural/Comparator). O(log n). add() retorna false se elemento já existe.
4. Low Coupling Minimizar dependências entre classes. Mudança em A não quebra B. Favorece reuso e testabilidade.	Map: HashMap : unordered LinkedHashMap : ordered TreeMap : sorted Métodos: put(k,v) get(k) containsKey(k) keySet()
5. High Cohesion Classe faz coisas fortemente relacionadas. Coesão baixa = classe faz 'tudo'. Alta coesão facilita manutenção.	ordered vs sorted: ordered = iteração previsível (ex: ordem de inserção) sorted = ordenação por regra natural ou Comparator
6. Polymorphism Variações de comportamento por tipo → use polimorfismo (@Override), não if/else com verificação de tipo.	Array vs Collection<Object> vs Collection<Tipo>: Array: tamanho fixo na declaração ← desvantagem Collection<Object>: dinâmico, mas sem segurança de tipo Collection<Tipo>: dinâmico + checagem em compilação ← ideal
7. Pure Fabrication Classe artificial (não existe no domínio) para manter coesão. Ex: RepositorioVenda (acesso a BD).	■ Exceções — 3 questões
8. Indirection Objeto intermediário para desacoplar dois elementos. Ex: Controller entre UI e domínio. → Suporta Low Coupling.	Hierarquia: Throwable → Error Exception Exception → RuntimeException (unchecked) + demais (checked)
9. Protected Variations Encapsular pontos de variação com interface estável. Mudanças internas não afetam clientes da interface.	Checked: deve declarar throws ou capturar. Sinaliza: recurso indisponível ou condição necessária ausente. Unchecked: RuntimeException ou Error. Erro de programação.
Relações entre princípios: Low Coupling ↔ High Cohesion (tensão natural) Indirection → suporta Low Coupling Pure Fabrication → aumenta High Cohesion Protected Variations → usa Polymorphism	Regras importantes: • finally: SEMPRE executa (exceto System.exit()) • Multicatch (Java 7): catch(Ex1 Ex2 e) • INCORRETO: 'checked não precisam ser especificadas' ← 4.3e
	Custom exception (padrão da prova): <pre>class MinhaEx extends Exception { private String campo; MinhaEx(String msg, String campo) { super(msg); // inicializa message this.campo = campo; } }</pre> Lançar: <code>throw new MinhaEx("msg", val);</code> Capturar: <code>catch(MinhaEx e) { e.getMessage(); }</code>
	Padrão prova2 — Albergue: <pre>1.1: extends Exception 1.2: private String nome; private String sobrenome; 1.3: super(mensagem); 1.4: throw new ExcecaoHospedeNaoEncontrado("Hospede nao encontrado", nome, sobrenome); 1.5: try { 1.6: catch(ExcecaoHospedeNaoEncontrado e) { 2.1: ArrayList<Reserva> r = new ArrayList<>(); 2.2: listaReservas.add(reserva);</pre>

■ DSS vs Diagrama de Sequência — 2q
DSS = Diagrama de Sequência do SISTEMA (análise) - Sistema = caixa preta (:Sistema) - Atores externos + operações de sistema - Mostra O QUE o sistema faz - Artefato de ANÁLISE / Requisitos
DS = Diagrama de Sequência (design) - Objetos internos VISÍVEIS - Mostra COMO o sistema faz internamente - Gerado a partir dos contratos de operação - Artefato de DESIGN / Projeto
Elementos do DSS: - Ator + :Sistema (":" + sublinhado = instância) - Seta sólida ■ = chamada de operação - Seta tracejada ·■ = valor de retorno loop[guarda] = moldura de iteração
Triades → operações de sistema: - Sistema solicita... Usuário informa → gera operacao(params) - Sistema responde... Nº de triades = Nº de operações de sistema
Nomes de operações: ✓ entrarItem(idItem, qtd) — verbo genérico ✓ criarNovaVenda() ✓ finalizarVenda() ✓ fazerPagamento(quantia) ✗ escanear() ← impl. tecnológica ✗ processarX() / tratarX() ← verbos vagos ✗ setItem() / getItem() ← padrão atributo - Iniciar com verbo. Bool → pergunta: foiVendaFinalizada() - Mesmo parâmetro NÃO deve aparecer em 2 operações!
Fluxo de artefatos PU: Modelo de Domínio → DSS → Contratos → Modelo de Projeto

■ Contrato de Operação — 1q
Seções obrigatórias: Operação: assinatura completa Responsabilidades: o que a operação faz Pré-condições: estado antes (assumido true) Pós-condições: estado após (voz passada!)
Pós-condições (sempre voz passada): - Instância de X <i>foi criada</i> - Associação entre A e B <i>foi formada</i> - Atributo X <i>foi modificado</i> para Y
Liga DSS ao Modelo de Projeto. Define O QUE, não COMO.
■ Modelos OO — Análise e Design — 2q
Análise OO: WHAT — conceitos do domínio do problema
Design OO: HOW — objetos de software e colaboração
Modelo de Domínio: - Classes conceituais (mundo real, NÃO software) - Normalmente SEM métodos - Substantivos do domínio → classes conceituais - Se X é número/texto no mundo real → atributo, não classe - Cenários de caso de uso → entrada para criar o MD

Ciladas da prova2: ✗ 4.8e: 'se X é número, é classe conceitual' ← ERRADO (é atributo) ✗ 4.9c: MD descreve objetos de software ← ERRADO (é mundo real) ✗ 4.9e: 'mais importante projetar do que entender UML' ← ERRADO
--

■ Generics — 1q
✓ NÃO aceita primitivos (Integer, não int) ✓ Checagem de tipo em tempo de compilação ✓ Elimina casts desnecessários ✓ Torna código reutilizável para vários tipos ✗ Type erasure: tipo genérico NÃO é mantido em runtime! ← 4.1d Diamond: <code>new ArrayList<>()</code>
■ Visibilidade para Design — 1q
Para A enviar mensagem a B, A precisa de visibilidade: Atributo: B é campo de A → persistente Ex: <code>this.venda.calcularTotal()</code> Parâmetro: B passado ao método → temporária Ex: <code>metodo(Venda v) { v.calcular(); }</code> Local: B declarado no método → temporária Ex: <code>Venda v = new Venda();</code> Global: B visível globalmente → persistente (raro)
■ Ciladas — Datas e Java SE 7
Pré-Java 8: <code>Date</code>, <code>GregorianCalendar</code>, <code>DateFormat</code> Java 8+: <code>LocalDate</code>, <code>LocalDateTime</code>, <code>DateTimeFormatter</code> <code>LocalDateTime.now()</code> = data/hora atual ✓ <code>LocalDate.of(y,m,d)</code> = data específica ✓ <code>localDate.plus(dias)</code> = somar dias ✓ ✗ 4.6e: Calendar herda de GregorianCalendar ← ERRADO (inverso!) ✗ 4.7c: <code>of</code> de <code>LocalDate</code> soma dias ← ERRADO (cria data)
Java SE 7 — novidades: ✓ Files: métodos estáticos para arquivos ✓ String em switch ✓ Multicatch: <code>catch(Ex1 Ex2 e)</code> ✓ Literais com underscore: <code>1_000_000</code> ✗ 4.5e: WatchService usa Adapter ← ERRADO